BUILD SPEC · INTERNAL

WTF OS

Phase 1 & Phase 2 — developer scope and checklist

SpaceBlanket.AI × WTF Podcast, Nikhil Kamath Media Group

Build owner	From	Client	Version
Shesh	Nikhai Jaysen, SpaceBlanket.AI	WTF Podcast	2.0 · 19 Aug 2026

WHAT WE'RE BUILDING

WTF runs 80+ long-form episodes across two YouTube channels, plus Shorts, Reels and six content IPs. Three problems:

– Editors spend 60–90 minutes per episode scrubbing manually for a moment they already remember.

– Published YouTube timestamps don't match the uncut files editors actually work in.

– Keyword search finds “health”. It doesn't find “the part where he talks about what drives him.”

WTF OS is one platform with one AI layer across all of it. The POC is live at `wtfmedia.vercel.app`.

Phase 1 turns that POC into a production platform where an editor types a topic and lands on the exact moment, in both the YouTube cut and the uncut file.

Phase 2 turns the platform from a search box into something that produces work — research briefs, cross-platform reporting, clip recommendations, and a shared production calendar.

> **HOW TO USE THIS DOCUMENT** Every line is a checkbox — unchecked means not done. **Done when** is the real definition of done, not the checklist above it. **Needed from WTF** blocks the module; don't start it, raise it. **[OPEN]** is an unresolved decision, listed in Section 7 — don't guess.

PHASE 1 · WEEKS 1–3

Podcast Brain

Content intelligence — search, transcripts, dual video tabs

> **GOAL** An authenticated, production-grade platform where any WTF team member asks a question in plain language and lands on the exact moment in the exact video, in whichever cut they're working from.

1.1 Platform foundation & access

Harden the POC into something the team can log into and rely on.

☐ Move `wtfmedia.vercel.app` from POC to production build — separate dev, staging and prod environments

☐ Authenticated access only. No route reachable without login

☐ WTF-only accounts — invite-based, admin-controlled, with deactivation

☐ Role model: `admin`, `editor`, `production`, `finance`, `management`

☐ Phase 1 only needs `admin` and `editor` to behave differently. Build the rest as empty shells so later phases don't force a migration

☐ Permissions enforced server-side, never only in the UI

☐ Audit log from day one: logins, searches, views, exports, admin actions

☐ Error tracking and structured application logging

☐ Domain purchased and pointed **[OPEN]**

☐ Deployment target provisioned **[OPEN]**

> **DONE WHEN** A WTF team member logs in with their own credentials, sees only what their role allows, and every action is traceable.

1.2 Episode ingestion

The backbone. Everything else reads from this.

☐ Pull all episodes from **both YouTube channels** — video ID, title, description, publish date, duration, thumbnail, chapter markers

☐ Ingest the **uncut files supplied directly by WTF** — no third-party tooling required, they hand these over

☐ Episode metadata schema: IP, channel, language, guest name(s), content bucket (`M1` / `S1` / `S2` / `ES`), publish date

☐ Six IPs stored as a first-class field: Nitya Kamat, BTS, POV, Animation, Stitched, Ways of the World

☐ Ingestion is idempotent — re-running must not duplicate or corrupt records

☐ Admin ingestion dashboard: what's indexed, what failed, why

☐ Async job processing with visible status. 80+ episodes will not ingest in one request

> **DONE WHEN** Every episode WTF has handed over is indexed, from both sources, with a status visible to admins.

1.3 Transcripts & timestamp reconciliation

> **READ THIS TWICE** **Transcripts for Phase 1 are pulled from the uncut version, not the published YouTube version.** The uncut file carries the complete context — animation cues, unedited passages, everything the published edit drops. It is the source of truth. The YouTube transcript exists only so the platform can also serve timestamps against the public video.

☐ Generate transcripts from the **uncut files** as the primary source

☐ Run our own ASR pass rather than relying on YouTube auto-captions — they're inconsistent on Indian English and unusable on Hindi

☐ Store transcripts as **segments**, never as a single blob. Each segment carries `start_ms`, `end_ms`, `speaker`, `text`, `source` (youtube | uncut)

☐ Speaker separation where feasible — “where the guest said X” vs “where Nikhil said X” will be asked for

☐ **Timestamp reconciliation:** compute and store the offset between the uncut file and the published YouTube version, per episode. Do not hardcode ~60s — measure it per episode by aligning transcripts

☐ Convert a timestamp in either direction, uncut to YouTube and back

☐ Episodes with no uncut file supplied: handle gracefully with an honest empty state, never an error

☐ Hindi episodes — transcript and search language handling **[OPEN]**

> **DONE WHEN** For 10 manually checked episodes, a moment found in the uncut transcript resolves to the correct YouTube timestamp within 2 seconds, and back again.

1.4 Dual video tabs — YouTube and Uncut

☐ Two tabs on every result and episode view: **YouTube** and **Uncut**

☐ YouTube tab: embedded player, seeks to the exact timestamp, no external tab switching

☐ Uncut tab: plays the uncut file from the exact timestamp

☐ Switching tabs preserves the moment — 14:32 on YouTube lands on the same frame in the uncut, using the stored offset

☐ Copy-timestamp button on both tabs. Editors will paste these into the sheets they already use

☐ Clean degradation when one of the two sources is missing

> **DONE WHEN** An editor finds a moment, watches it on YouTube, switches to Uncut, and it's the same moment.

1.5 Search

The feature the whole platform is judged on.

☐ Keyword and exact-phrase search across all transcript segments

☐ **Semantic search** — a query for “trauma” surfaces a passage about a hard childhood that never uses the word; “philosophy” surfaces how someone thinks about meaning. Concept-level retrieval, not string matching

☐ Hybrid ranking — semantic recall combined with keyword precision, not one or the other

☐ Result object: episode title · IP · channel · timestamp · transcript snippet with the match highlighted · relevance score · inline player

☐ Filters: episode, IP, guest, channel, date range, language

☐ Multiple results per query, default 10, paginated. Editors want options, not one answer

☐ Low-confidence handling — say “nothing strong here” rather than returning noise

☐ Query logging: latency, tokens in/out, cost per query, attributable to user and feature

> **DONE WHEN** Against a 20-query evaluation set written by WTF's editorial team, including at least 8 concept-level queries, the correct moment appears in the top 3 results at least 80% of the time. Median latency under 3 seconds.

> **BUILD NOTE** Create the evaluation set *before* tuning retrieval. Without it, “is the search good enough?” becomes an argument instead of a number.

1.6 Performance dashboard

YouTube only in Phase 1. Instagram and LinkedIn arrive in Phase 2.

☐ Subscriber count, views, watch time, likes, comments — channel level, both channels

☐ Per-episode performance view

☐ Trend over time, plus a simple record-broken flag (best 24h / 7d / 30d)

☐ Scheduled refresh, served from cache. Do not call the API on page load

☐ Hindi content tracking **[OPEN]**

> **DONE WHEN** WTF sees both channels in one place, refreshed at least daily, without opening YouTube Studio.

Phase 1 — needed from WTF

#	What	Blocks	By
1	YouTube account access for our email ID — Data API + Analytics API scopes, both channels	1.2, 1.6	Week 1
2	Uncut files handed over, with a clear episode-to-file mapping	1.2, 1.3, 1.4	Week 1
3	Full episode list — count, IP tagging, language per episode	1.2	Week 1
4	20-query editorial evaluation set with expected answers	1.5	Week 2

#	What	Blocks	By
5	Team roster and roles for account provisioning	1.1	Week 2
6	Domain name approved	1.1	Week 2

PHASE 1 IS DONE WHEN Three editors use the platform instead of scrubbing, without being reminded to, and clip discovery drops below 2 minutes per clip.

PHASE 2 · WEEKS 3–7

AI agent layer

Research and automation — briefs, metrics, suggestions, calendar

GOAL Move from retrieval to output. The platform stops answering questions and starts producing work.

2.1 Research agent — Snacks, Appetizers, Main course, Desserts

How research works today

- **~40% offline.** Direct discussion with industry leaders, founders, CEOs and VCs on the specific topic or industry. This stays human. The agent does not touch it.
- **~60% desk research**, spread across Claude, NotebookLM, Perplexity, and manual listening to the guest's past podcast appearances. Fragmented across tools, and nothing carries over from one episode to the next.
- Total: roughly one week per episode.

THE AGENT REPLACES THE 60% One request inside WTF OS, one cited brief out. No tool-hopping and no manual searching.

2.1.1 Input and output

Input — any one of these is enough: **guest name**, **company name**, or **industry**. Output — one brief, four tiers, every claim cited.

Tier	Output
Snacks	Guest personality profile — who they are, expanded personal background
Appetizers	Core industry analysis — share of GDP, comparison against similar industries
Main course	Exact industry positioning — what a 20–24-year-old in India could build in this industry
Desserts	Biggest takeaway, and where the industry is heading

☐ Accept any single input, resolve it to an entity, ask for disambiguation only when genuinely ambiguous

☐ Auto-expand the chain: guest resolves to company resolves to industry, without the user typing all three

☐ Brief maps to the four tiers, in order

☐ Regenerate a single tier without redoing the whole brief

☐ Editable in-platform, versioned, attached to the episode record

☐ Team's offline notes — the 40% — uploaded against the guest record, included in the agent's context, and weighted **above** web material as a primary source

☐ Export target **[OPEN]**

2.1.2 Source layer

☐ **Live web research with citations — Perplexity Sonar API.** Every Sonar response is search-grounded and returns source URLs; Sonar Pro returns roughly double the citations of standard Sonar. Use search domain filters to keep sourcing credible

☐ **Podcast corpus.** Find and ingest every prior podcast, interview and long-form appearance by the guest. Discovery through YouTube search, podcast RSS feeds, and the Apple/Spotify catalogues. Transcripts from existing captions where available, otherwise our own ASR pass — the same pipeline as 1.3

☐ Podcast corpus is searchable and quotable. The brief should be able to state what the guest has already said, where, and when. Two uses: raw material, and knowing which questions have been asked to death

☐ **WTF's own episode library.** Reuse the Phase 1 retrieval layer. If a past WTF episode touched this industry or guest, the brief says so

☐ Structured data for GDP contribution, market size and comparables. Prefer primary sources — government statistics, industry bodies, company filings — over aggregators and listicles

☐ Every source stored with URL, publisher, publication date and retrieval date

> **NOTEBOOKLM — READ BEFORE PROMISING ANYTHING** There is no public consumer API for NotebookLM (now renamed Gemini Notebook). Google's programmatic access is enterprise-only via Gemini Notebook Enterprise, with notebook and audio-overview operations still in pre-GA preview; the separate standalone podcast endpoint is deprecated and not allowlisting new customers. We cannot integrate the tool the research team uses today. What we replicate is the *function*, not the product — grounded question-and-answer over a fixed source set, with citations, built inside WTF OS on the Phase 1 retrieval layer over the guest's corpus. Say this to WTF plainly rather than implying a NotebookLM connection exists.

2.1.3 Multi-agent orchestration

One orchestrator decomposes the brief into parallel tracks — one per tier, plus one per source type — runs them concurrently, then synthesises and verifies.

☐ Orchestrator plus parallel sub-agents. Not one long sequential chain

☐ A sub-agent per tier: Snacks, Appetizers, Main course, Desserts

☐ Dedicated sub-agents for podcast-corpus retrieval and for numeric verification

☐ Live job status while it runs — the team should see which tracks have finished

☐ Per-track retries and failure handling. One dead track must not kill the brief

☐ Provider-agnostic orchestration interface. Do not hard-wire to a single vendor

☐ Evaluate **Kimi Agent Swarm** as the orchestration layer **[OPEN]**

On Kimi Agent Swarm. Moonshot AI's Agent Swarm is a self-organising multi-agent system — the model decomposes the task and spawns its own sub-agents rather than requiring hand-built orchestration through LangGraph or CrewAI. K2.5 (January 2026) spawned up to 100 sub-agents; K2.6 (April 2026) raised that to 300 sub-agents and roughly 4,000 coordinated steps, with Moonshot reporting around 4.5x speedup over single-agent runs on complex tasks. It is exposed through an OpenAI-compatible endpoint, and the weights are open under a modified MIT licence, so self-hosting is on the table.

Two cautions, both directly relevant to this build:

– **Reliability.** Tool-call failures in agentic loops ran at roughly 12% at K2.5, and the swarm shipped labelled Beta. Assume failures and build for them.

– **Citations.** Reviewers have specifically flagged that swarm output tends to be under-cited and stated too absolutely. That is precisely our hardest requirement. Citation enforcement sits in our layer, never the orchestrator's. Do not trust the model to cite honestly.

> **BEFORE COMMITTING** Run a bake-off on the same three guests — Kimi swarm against our own orchestration — scored on citation density, factual accuracy, latency and cost per brief.

2.1.4 Citations and verification

Not a nice-to-have. The brief feeds questions asked on camera — a wrong number in a brief becomes a wrong number in an episode.

☐ Every factual claim carries an inline citation: source name, URL, date

☐ Claims the agent cannot source are dropped or explicitly flagged unverified. Never silently included

☐ Claims drawn from a podcast cite the episode **and timestamp**, not just the show name

☐ Separate verification pass across all numbers — GDP figures, market sizes, funding amounts — checked against a second independent source

☐ Deduplicated source list at the end of every brief, grouped by tier

☐ Where two sources disagree, surface the disagreement rather than silently picking one

> **DONE WHEN** For three real upcoming guests, the research team rates the brief as a usable starting point that saves at least half a day of desk research; every factual claim carries a working citation; and the podcast corpus surfaces at least five prior appearances per guest where they exist.

2.2 Cross-platform metrics

☐ Instagram analytics integration

☐ LinkedIn analytics integration

☐ Unified dashboard: YouTube, Instagram and LinkedIn side by side, metrics normalised

☐ Breakdown by IP and by content bucket (`M1` / `S1` / `S2` / `ES`)

☐ Scheduled refresh, cached

> **DONE WHEN** One dashboard shows all three platforms, filterable by IP and content bucket, refreshed daily.

2.3 Content performance analysis & clip suggestions

☐ Correlate clip and Short performance against transcript features — topic, guest, length, hook

☐ Surface what actually performs: which topics, which guests, which formats

☐ **Predictive clip suggestions** — for a newly ingested episode, propose ranked candidate moments for Shorts, each with a stated reason

☐ Editor feedback loop — thumbs up/down on each suggestion, stored and used to improve ranking

☐ Trigger mode **[OPEN]**

> **DONE WHEN** For a new episode the agent proposes 10 ranked Shorts candidates and the editorial team agrees at least 5 are worth cutting.

> **FRAMING FOR THE CLIENT** This is a recommender working on a small dataset. Sell it as “a strong shortlist to start from”, never “it knows what will go viral.”

2.4 Automated internal reporting

☐ Weekly content performance summary — generated and delivered without anyone touching it

☐ Monthly rollup with trend commentary

☐ Anomaly and record-break alerts

☐ Configurable recipients and delivery channel **[OPEN]**

☐ Reports also viewable in-platform

> **DONE WHEN** The weekly report lands on schedule for three consecutive weeks with no manual intervention.

2.5 Production calendar

☐ Shared calendar visible to the whole WTF team

☐ Editable by team members with the right role

☐ Colour coding across shoot start dates, posting times, milestones, and per IP

☐ Month view, week view, and a per-IP filter

☐ Entries link back to the episode record and its content buckets

☐ Notifications ahead of shoots and posting slots

☐ Native in-platform vs. Google Calendar sync **[OPEN]**

> **DONE WHEN** WTF plans a full month of shoots and posts inside WTF OS and stops maintaining the parallel sheet.

Phase 2 — needed from WTF

#	What	Blocks
7	Instagram and LinkedIn analytics access	2.2
8	Walkthrough of how episode research happens today — who does it, and with which tools	2.1
8b	Sample offline research notes from the 40% human layer, so the agent knows what it is supplementing	2.1
9	Two or three sample research briefs from past episodes, for format benchmarking	2.1
10	Historical Shorts performance data	2.3
11	Current shoot and posting calendar, plus their colour convention	2.5

> **PHASE 2 IS DONE WHEN** The research team's pre-production brief starts as an AI draft rather than a blank page, and the weekly content report arrives without anyone building it.

SECTION 5

Technical standards

Applies across both phases — not optional

Security

☐ Role-based access enforced server-side

☐ Assume anything ingested may be guest-confidential. Treat it that way

☐ Secrets in a managed store, never in the repo

☐ All API tokens rotatable without a redeploy

Cost and observability

☐ Token usage logged per query and per agent run, attributable to a user and a feature

☐ The monthly AI consumption report is a commitment made to WTF. Build the data collection for it now, not at month-end

☐ Cost ceiling alerting

Performance

☐ Search median under 3s, p95 under 6s

☐ Dashboards served from cache

☐ Ingestion async with visible job status

Engineering hygiene

☐ Separate dev, staging and prod databases

☐ Migrations, not manual schema edits

☐ README covering local setup, environment variables and the ingestion runbook

☐ Demo build on staging every Friday, regardless of state

6. Data model

Indicative, not prescriptive. Adjust as needed, but these entities and relationships need to exist.

```
users            id, email, name, role, status, created_at

episodes         id, youtube_video_id, channel (primary|secondary), ip,
                 title, description, published_at, duration_ms,
                 language, guests[], content_bucket, thumbnail_url

sources          id, episode_id, type (youtube|uncut), uri,
                 duration_ms, offset_vs_youtube_ms

segments         id, episode_id, source_id, start_ms, end_ms,
                 speaker, text

embeddings       segment_id, vector, model_version

metrics_daily    id, platform, entity_type (channel|episode|clip),
                 entity_id, date, views, likes, comments,
                 followers, watch_time

research_briefs  id, episode_id, guest, tier_snacks, tier_appetizers,
                 tier_main, tier_desserts, sources[], version,
                 status, created_by

clip_suggestions id, episode_id, start_ms, end_ms, rationale,
                 score, editor_feedback

calendar_events  id, episode_id, ip, type (shoot|post|milestone),
                 starts_at, ends_at, colour, owner

query_log        id, user_id, query, mode, latency_ms,
                 tokens_in, tokens_out, cost, result_ids[]

audit_log        id, user_id, action, entity, entity_id, at, meta
```

> **DESIGN CONSTRAINT** Keep the retrieval layer source-agnostic. A segment should not care whether it came from an episode transcript or from something ingested later. Building that assumption in now avoids a rewrite.

7. Open items

Resolve before the dependent work starts.

#	Item	Owner	Blocks
1	**Deployment target — WTF's ZTV cloud server plus local NAS, or our cloud.** Their kickoff minutes committed to ZTV + NAS; the POC runs on Vercel. Nikhai and Shesh to check specs, access and feasibility together. This decides the architecture, not just the hosting bill	Nikhai + Shesh	1.1, all
2	Domain name — WTF to shortlist and approve	WTF	1.1
3	Hindi content: separate channel or YouTube auto-dubbing? Determines what analytics exist and how transcripts are handled	WTF	1.3, 1.6
4	Final role list and permission matrix — who maps to which of the five roles	WTF	1.1
5	Perplexity Sonar tier (Sonar vs Sonar Pro vs Deep Research) and a monthly ceiling for research spend	Nikhai	2.1
6	Research brief export target — Google Doc, Notion, or in-platform only	WTF	2.1
7	Clip suggestions: automatic on ingest, or editor-triggered	WTF editorial	2.3
8	Report delivery channel and recipient list	WTF	2.4
9	Calendar: native in-platform or Google Calendar sync	WTF	2.5
10	Kimi Agent Swarm as the orchestration layer, or our own orchestration. Decide after the bake-off, not before	Nikhai + Shesh	2.1
11	Podcast corpus scope — how far back, which platforms, and where transcripts are stored	WTF + Shesh	2.1

8. Not in Phase 1 or Phase 2

Real work, scoped separately. Don't build it now — but don't architect in a way that makes it painful later.

– Vendor payments, budget approvals, travel requests, venue credential management

– Retroactive timestamp correction for the earliest episodes with no uncut files

– CRM and guest pipeline management

– WhatsApp group import, email integration, and cross-search across episodes, emails and documents

– Archival and stock footage sourcing

– Guest contract workflow automation

> **SCOPE BOUNDARY** Anything a WTF team member asks for that isn't in Sections 1 to 6 of this document is a change request. Route it to Nikhai.

9. First five things to do

1. Read Phase 1 and Section 7. Flag anything unbuildable as written.
2. Estimate against the Phase 1 modules — where does the three-week timeline actually break?
3. Settle open item 1 with Nikhai. Deployment decides the architecture.
4. Stand up 1.2 ingestion for YouTube first, so progress isn't hostage to file handover.
5. Set up staging with a Friday demo build before writing feature code.

> Questions go to Nikhai. Don't build around an open item — raise it.